

Chapter 6, Section F: Higher-Order Functions (Functions That Take/Return Functions)

1. What a higher-order function is

A **higher-order function** is any function that does **at least one** of these:

- takes another function **as an argument**, or
- **returns** a function

This is only possible because functions are first-class values (Section E) — they can be passed around like any other value.

```
function higherOrder(fn) {  // takes a function as argument
  fn();
}

function factory() {
  return function () { };  // returns a function
}
```

Both `higherOrder` and `factory` are higher-order functions.

2. The function passed in/out is just called a “regular” function

To avoid confusion: a function that does **not** take or return another function is just a regular (first-order) function.

```
function add(a, b) {
  return a + b;  // returns a NUMBER, not a function -- not higher-
order
}
```

3. Type 1 - Takes a function as an argument

```
function greetUser(name, formatter) {
  console.log(formatter(name));
}

function upperCase(str) {
  return str.toUpperCase();
}

greetUser("Aniket", upperCase); // "ANIKET"
```

`greetUser` doesn't know or care what `formatter` does internally – it just calls it. This is powerful because you can swap `formatter` for different behavior without touching `greetUser` at all:

```
function lowerCase(str) {
  return str.toLowerCase();
}

greetUser("Aniket", lowerCase); // "aniket"
```

Real examples you already know: `.map()`, `.filter()`, `.forEach()`, `addEventListener()` – all take a function as an argument.

```
[1, 2, 3].map(function (n) {
  return n * 2;
});
```

`.map` is the higher-order function here – it takes your function and calls it on each array element.

4. Type 2 - Returns a function

```
function makeGreeter(greeting) {
  return function (name) {
    console.log(`${greeting}, ${name}`);
  };
}

const sayHello = makeGreeter("Hello");
const sayHi = makeGreeter("Hi");

sayHello("Aniket"); // "Hello, Aniket"
sayHi("Rahul");     // "Hi, Rahul"
```

`makeGreeter` is a “function factory” – it builds a specific function based on what you give it, and hands that new function back. `sayHello` and `sayHi` are each separate functions, remembering a different greeting.

(This remembering behavior is a **closure**, covered next in Section G – higher-order functions and closures work together constantly.)

5. A function can be both at once

```
function withLogging(fn) {           // takes a function
  return function (...args) {       // returns a function
    console.log("calling function...");
    return fn(...args);
  };
}

function add(a, b) {
  return a + b;
}

const loggedAdd = withLogging(add);
loggedAdd(2, 3);
// prints: "calling function..."
// returns: 5
```

`withLogging` takes `add` in, and returns a **new, wrapped version** of it that adds logging behavior. This pattern (wrapping a function to add extra behavior) is common in real codebases – it’s the basis of things like middleware in Express.js.

6. Why higher-order functions matter

Benefit	Explanation
Reusability	One function (<code>greetUser</code> , <code>.map()</code>) can work with many different behaviors, just by swapping the function passed in
Avoids repeating code	Instead of writing a near-identical loop for every transformation, you write one loop and pass in different logic
Composability	Small functions can be combined/wrapped to build bigger behavior (point 5)
Foundation of functional programming style	Most array methods, async code, and modern JS patterns rely on this

7. Built-in higher-order functions you already use (or will soon)

```
array.map(fn)      // transforms each element
array.filter(fn)   // keeps elements that pass a test
array.forEach(fn)  // runs fn on each element
array.reduce(fn)   // combines elements into one value
setTimeout(fn, ms) // runs fn later
```

Every one of these takes a function as an argument - meaning every one is a higher-order function. You've actually been using higher-order functions since Section B's `reduce` example, without the label.

8. Summary

- **Higher-order function** = a function that takes a function as an argument, returns a function, or both.
- This only works because functions are first-class values (Section E).
- Two shapes: "takes a function in" (e.g. `.map()`, `greetUser(name, formatter)`) and "returns a function out" (e.g. `makeGreeter`).
- It's how JavaScript avoids rewriting similar logic repeatedly - swap the function passed in instead of rewriting the whole thing.
- Nearly all array methods and async functions (`setTimeout`, event listeners) are higher-order functions.

9. Practice question

```
function multiplyBy(factor) {
  return function (n) {
    return n * factor;
  };
}

const double = multiplyBy(2);
console.log(typeof multiplyBy);
console.log(typeof double);
console.log(double(10));
```

Answer:

1. `multiplyBy` is a named function declaration, so `typeof multiplyBy` is `"function"`.
2. `multiplyBy(2)` runs and returns a new function (not a number) - `double` holds that returned function, so `typeof double` is also `"function"`.
3. `double(10)` calls that inner function with `n = 10`, using the remembered `factor = 2`, giving $10 * 2 = 20$.

Output:

```
function
function
20
```